

Intergiciels

Examen

Daniel Hagimont

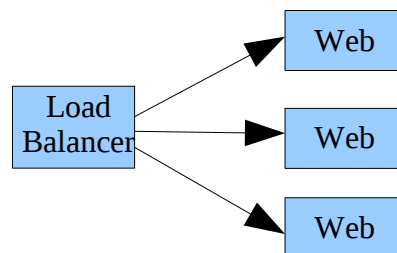
Durée: 1 heures 45 minutes, documents autorisés

*Lire l'ensemble des énoncés avant de commencer à répondre. La **clarté**, la **précision** et la **concision** des réponses, ainsi que leur **présentation matérielle**, seront des éléments importants d'appréciation. Donnez essentiellement les programmes Java demandés. Dans vos programmes, vous n'avez pas à programmer les imports et le traitement des exceptions.*

PROBLÈME I (sockets)

Vous avez à réaliser avec des sockets en Java un répartiteur de charge (une classe *LoadBalancer*) qui reçoit des requêtes HTTP et les distribue de façon aléatoire vers un ensemble de serveurs Web. Le début de la classe *LoadBalancer* est le suivant :

```
public class LoadBalancer {  
    static String hosts[] = {"host1", "host2"};  
    static int ports[] = {8081,8082};  
    static int nbHosts = 2;  
    static Random rand = new Random();  
    ...  
}
```



A chaque réception d'une requête HTTP, *LoadBalancer* transfère la requête à un des serveurs web (les adresses de ces serveurs sont données par les tables *hosts* et *ports*) et *LoadBalancer* transfère le résultat de la requête à l'émetteur. Le choix du serveur Web est aléatoire (*rand.nextInt(nbHosts)* retourne un entier entre 0 et *nbHosts-1*). Pour être efficace, *LoadBalancer* est évidemment multi-threadé.

Pour la programmation des entrées/sorties, on utilisera :

- *InputStream*
 - *public int read(byte[] b);* // bloquante, retourne le nombre de bytes lus
- *OutputStream*
 - *public void write(byte[] b, int off, int len);* // écrit les len bytes à la position off

et on suppose que les requêtes et réponses sont lues ou écrites en un seul appel de méthode avec un buffer de 1024 bytes.

PROBLÈME II (RMI)

Vous devez réaliser avec Java RMI une implantation d'un intergiciel de type JMS. Ce service est implanté sous la forme d'un serveur RMI (sur une machine serveur) qui implante l'interface *MOM*, qui fournit 2 méthodes ayant les prototypes suivants :

```
public void publish(String topic, Message message) // publie un message sur un topic  
public void subscribe(String topic, Callback cb) // s'abonne à un topic
```

L'abonnement (*subscribe*) à un topic prend en paramètre une référence à un objet d'interface *Callback* permettant le rappel du client pour lui transmettre un message publié sur le topic. L'interface *Callback* fournit la méthode de prototype :

```
public void onMessage(Message message)
```

Vous devez programmer ce service (interface *MOM* et classe *MOMImpl*) ainsi qu'un exemple de programme client démontrant l'utilisation du service lors d'une publication et d'un abonnement.

Pour simplifier, je vous propose d'implanter une classe *MOMImpl* qui ne prend en compte qu'un unique topic de nom "topic".

```

1 // Problème 1
2
3 public class LoadBalancer extends Thread {
4     static String hosts[] = {"host1", "host2"};
5     static int ports[] = {8081, 8082};
6     static int nbHosts = 2;
7     static Random rand = new Random();
8     Socket cli;
9
10    public LoadBalancer (Socket s) {
11        cli = s;
12    }
13
14    public static void main (String args[]) {
15        ServerSocket ss = new ServerSocket(8080);
16        while (true) {
17            Socket s = ss.accept();
18            // Important de ne pas faire .run() !!!
19            new LoadBalancer(s).start();
20        }
21    }
22
23    public void run() {
24        try {
25            int nServ = rand.nextInt(nbHosts);
26            Socket serv = new Socket(hosts[nServ], ports[nServ]);
27            InputStream cliIS = cli.getInputStream();
28            InputStream serv_is = serv.getInputStream();
29            OutputStream serv_os = serv.getOutputStream();
30            byte buff[] = new byte[1024];
31
32            int nb = cli_is.read(buff,0,1024);
33            serv_os.write(buff,0,nb);
34            nb = serv_is.read(buff,0,1024);
35            cli_os.write(buff,0,nb);
36            cli.close();
37            serv.close();
38        } catch (IOException e) {
39            e.printStackTrace();
40        }
41    }
42 }
43
44
45
46 // Problème 2
47 public interface Callback extends Remote {
48     public void onMessage(Message m) throws RemoteException;
49 }
50
51 public class CallbackImpl extends UnicastRemoteObject implements Callback {
52     public CallbackImpl() throws RemoteException {}
53 }

```

```

54     public void onMessage(Message m) throws RemoteException {
55         System.out.println(m);
56     }
57 }
58
59 public interface MOM extends Remote {
60     public void publish(String topic, Message m) throws RemoteException;
61     public void subscribe(String topic, Callback callback) throws RemoteException;
62 }
63
64 public class MOMImpl extends UnicastRemoteObject implements MOM {
65     Hashtable<String, ArrayList<Callback>> table;
66
67     public MOMImpl() throws RemoteException {
68         table = new Hashtable<String, ArrayList<Callback>>();
69     }
70
71     public void subscribe(String topic, Callback callback) throws RemoteException {
72         ArrayList<Callback> l = table.get(topic);
73         if (l == null) {
74             l = new ArrayList<Callback>();
75             table.put(topic, l);
76         }
77         l.add(callback);
78     }
79
80     public void publish(String topic, Message m) throws RemoteException {
81         ArrayList<Callback> l = table.get(topic);
82         for (Callback callback : l) {
83             callback.onMessage(m);
84         }
85     }
86
87     public static void main (String args[]) {
88         LocalRegistry.createRegistry(8081);
89         Naming.bind("//localhost:8081/mom", new MOMImpl());
90     }
91 }
92
93 public class Client {
94     public static void main (String args[]) {
95         // Important de caster en MOM et pas MOMImpl
96         MOM m = (MOM)Naming.lookup("//localhost:8081/mom");
97         m.subscribe("sujet1", new CallbackImpl());
98         m.publish("sujet1", "Un petit message :)");
99     }
100 }
101

```